PROGRAMMING PRINCIPLES CAT 3

MARKING SCHEME

Question 1 (2 marks)

Answer	Marks
OOP is a programming paradigm/approach that organises software around objects rather than functions and logic.	1
An object bundles together data (attributes/properties) and the methods (functions/behaviour) that operate on that data.	1
Total	**2**

Question 2 (3 marks)

Answer	Marks
Class: a blueprint/template that defines the attributes and methods of a type of object.	1
Object: an instance of a class, a real entity created from the blueprint, with its own values.	1
Valid real-world example, e.g. class Car (blueprint); objects myCar or Toyota (specific cars with their own colour and model).	1
Total	**3**

Question 3 (3 marks)

Award 1 mark for each correct comparison.

Aspect	Procedural Programming	Object-Oriented Programming	Marks
Focus	Functions/procedures and the sequence of steps	Objects and their interactions	1
Handling of data	Data is global/separate from functions and less protected	Data is bundled with methods and protected through encapsulation	1
Reuse	Limited reuse (mainly by copying/calling functions)	High reuse through inheritance and classes	1
Total			**3**

Question 4 (4 marks)

Answer	Marks
Encapsulation: bundling data and methods in a class and restricting direct access to the data (data hiding), usually using private members.	1

Valid example, e.g. a BankAccount class with a private balance accessed only through deposit() and getBalance().	1
Polymorphism: the ability of one interface/function/method to take many forms (same name, different behaviour).	1
Valid example, e.g. a function draw() behaving differently in Circle and Square classes (overriding), or overloaded functions with different parameters.	1
Total	**4**

Question 5 (4 marks)

Award 1 mark for each correct point, up to 2 advantages and 2 disadvantages.

Advantages (any 2)	Marks
Code reusability through inheritance	1
Data security through encapsulation	
Easier maintenance and modification	
Models real-world problems naturally	
Easier to manage large, complex programs	
Supports teamwork through modular classes	1

Question 6 (4 marks)

Disadvantages (any 2)	Marks
Steeper learning curve/more complex to learn	1
Programs can be larger and slower than procedural ones	
Requires careful design and planning	
Not suitable for very small, simple programs	
Can lead to over-engineering	1
Total	**4**

1 mark each.

Part	Purpose	Marks
a) #include <iostream>	Includes the input/output stream library so that cin and cout can be used	1
b) using namespace std;	Allows use of standard library names (e.g. cout, cin) without writing the std:: prefix	1
c) int main()	The main function where program execution starts	1
d) return 0;	Ends the main() function and returns 0 to the operating system, showing successful execution	1
Total		**4**

Question 7 (4 marks)

Award **1 mark** for each error correctly identified **and** corrected (any four).

Error	Correction	Marks
Missing semicolon after using namespace std	using namespace std;	1
Wrong operator in the input statement: cin << age;	cin >> age;	1
Wrong operator in the output statement: cout >> "Age is "	cout << "Age is "	1
Wrong operator before the variable: >> age	<< age; (full line: cout << "Age is " << age;)	1
Total		**4**

Corrected program:

```
#include <iostream>
using namespace std;

int main() {
   int age;
   cin >> age;
   cout << "Age is " << age;
   return 0;
}
```

Question 8 (6 marks)

a) Access specifiers (3 marks)

Answer	Marks
public: members can be accessed from anywhere, inside and outside the class.	1
private: members can be accessed only within the class (and its friends); this is the default for a class.	1
protected: members can be accessed within the class and by its derived (child) classes, but not from outside.	1
Subtotal	**3**

b) Evaluation of a public balance (3 marks)

Answer	Marks
Evaluation: the decision is poor/unsafe because any part of the program can read or change the balance directly.	1
Reason: it breaks encapsulation and data security, since there is no validation (e.g. the balance could be set to a negative or invalid value).	1
Recommendation: declare balance as private and provide public methods (getters/setters such as getBalance(), deposit(), withdraw()) that validate values.	1
Subtotal	**3**

Question 8 Total 6

Question 9 (5 marks)

a) Book class (3 marks)

Answer	Marks
Correct class declaration with the public: access specifier and closing semicolon after the class body	1
Correct attributes: string title; and int price;	1
Correct display() method printing both title and price using cout	1
Subtotal	**3**

Model answer:

```
class Book {
public:
   string title;
   int price;

   void display() {
      cout << "Title: " << title << endl;
      cout << "Price: " << price << endl;
   }
};
```

b) Statements in main() (2 marks)

Answer	Marks

Creates an object of the class (e.g. Book b1;) and assigns values to its attributes using the dot operator (e.g. b1.title = "C++"; b1.price = 500;)	1
Calls the method correctly (e.g. b1.display();)	1
Subtotal	**2**

Model answer:

```
Book b1;
b1.title = "Learning C++";
b1.price = 500;
b1.display();
```

Question 9 Total 5

Question 10 (5 marks)

Answer	Marks
Includes the necessary header(s) and namespace (#include <iostream>, using namespace std;) and has a valid main() function	1
Correct class Rectangle declaration (with public: and closing semicolon)	1
Method calculateArea(int length, int width) with the correct return type (int) and returns length * width	1
In main(): creates a Rectangle object, calls the method with values, and stores the result in a variable	1
Displays the result correctly using cout	1

Total	**5**

Model answer:

```
#include <iostream>
using namespace std;

class Rectangle {
public:
   int calculateArea(int length, int width) {
      return length * width;
   }
};

int main() {
   Rectangle rect;
   int area = rect.calculateArea(8, 5);
   cout << "Area = " << area << endl;
   return 0;
}
```

Expected output with the values above: Area = 40

MARK SUMMARY

Question	Topic	Marks
1	Definition of OOP and objects	2
2	Class vs object	3
3	Procedural vs OOP	3
4	Encapsulation and polymorphism	4
5	Advantages and disadvantages of OOP	4
6	Parts of a C++ program	4
7	Debugging a C++ program	4
8	Access specifiers and evaluation	6
9	Book class and objects	5
10	Rectangle program	5
	TOTAL	**40**